

Introducción a la Programación I

Clase 8: Diccionarios, Debug y Herramientas Iterables

Agenda

- **Diccionarios**

- ¿Qué es un diccionario?
- Operaciones básicas: acceder, agregar, eliminar
- Métodos e iteración

- **Debug**

- ¿Qué es el debugging?
- Tipos de errores en Python
- Herramientas de debugging

- **Herramientas Iterables**

- Iterables y colecciones
- `range()`, `enumerate()` y List Comprehension

¿Qué es un diccionario?

Un **diccionario** es una estructura de datos que almacena **pares clave-valor**.

- Cada **clave** es única y apunta a un **valor** específico.
- Permiten recuperar, actualizar y eliminar valores rápidamente a partir de su clave.

💡 Pensalo como un diccionario real: buscás la palabra (clave) y encontrás su definición (valor).

```
"AEP"   → "Aeroparque Jorge Newbery"  
"EZE"   → "Aeropuerto Internacional Ministro Pistarini"
```

Claves y valores

Claves

- Pueden ser **strings, números, booleanos** o cualquier tipo *comparable*.
- Deben ser **únicas** dentro del diccionario.
- No pueden ser listas u otros objetos mutables.

Valores

- Pueden ser de **cualquier tipo**: strings, números, listas, otros diccionarios, objetos, etc.
- No necesitan ser únicos.

Diccionarios en Python

Python llama `dict` a sus diccionarios. Se definen con llaves `{}`.

```
# Diccionario vacío
estudiante_edades = dict()
# o equivalentemente:
estudiante_edades = {}

# Diccionario con datos
edades = {
    "Micaela": 18,
    "Francisco": 19,
    "Mirtha": 22,
    "José": 27
}
```

Ejemplos de diccionarios

```
# Edades de estudiantes
```

```
edades = {  
    "Micaela": 18,  
    "Francisco": 19,  
    "Mirtha": 22,  
    "José": 27  
}
```

```
# Nombres de aeropuertos
```

```
nombre_aeropuerto = {  
    "AEP": "Aeroparque Jorge Newbery",  
    "EZE": "Aeropuerto Internacional Ministro Pistarini",  
    "CPC": "Chapelco – Aviador Carlos Campos"  
}
```

Acceder a valores

Para acceder al valor asociado a una clave se usa la **notación de corchetes**:

```
print(nombre_aeropuerto["AEP"])  
# Aeroparque Jorge Newbery  
  
print(edades["Mirtha"])  
# 22
```

⚠ Si la clave no existe, Python lanza un `KeyError`.

Para evitarlo se puede usar el método `.get()`:

```
print(edades.get("Carlos", "No encontrado"))  
# No encontrado
```

Agregar y actualizar entradas

Se usa el **operador de asignación** `=` para agregar o actualizar:

```
notas = {"Alice": 85, "Bob": 70}

# Agregar una entrada nueva
notas["David"] = 88

# Actualizar una entrada existente
notas["Alice"] = 95

print(notas)
# {'Alice': 95, 'Bob': 70, 'David': 88}
```

- Si la clave **ya existe** → el valor se **actualiza**.
- Si la clave **no existe** → la entrada se **crea**.

Eliminar entradas

Usando `del`

```
del notas["Bob"]  
print(notas)  # {'Alice': 95, 'David': 88}
```

Usando `pop()`

```
valor = notas.pop("David")  
print(valor)  # 88 (devuelve el valor eliminado)  
print(notas)  # {'Alice': 95}
```

`pop()` es útil cuando necesitas el valor antes de eliminarlo.

Métodos útiles de los diccionarios

Método	Descripción	Retorna
<code>.keys()</code>	Todas las claves	<code>dict_keys</code>
<code>.values()</code>	Todos los valores	<code>dict_values</code>
<code>.items()</code>	Todos los pares clave-valor	<code>dict_items</code>
<code>.get(k, default)</code>	Valor de la clave <code>k</code> o default	valor
<code>.pop(k)</code>	Elimina y retorna el valor de <code>k</code>	valor

`keys()` — Las claves del diccionario

```
notas = {  
    "Alice": 85,  
    "Bob": 70,  
    "Charlie": 60  
}  
  
print(notas.keys())  
# dict_keys(['Alice', 'Bob', 'Charlie'])  
  
# Convertir a lista  
lista_claves = list(notas.keys())  
print(lista_claves)  
# ['Alice', 'Bob', 'Charlie']
```

`values()` — Los valores del diccionario

```
notas = {  
    "Alice": 85,  
    "Bob": 70,  
    "Charlie": 70  
}  
  
print(notas.values())  
# dict_values([85, 70, 70])  
  
# Los valores pueden repetirse (las claves no)
```

`items()` — Pares clave-valor

```
notas = {  
    "Alice": 85,  
    "Bob": 70,  
    "Charlie": 60  
}  
  
print(notas.items())  
# dict_items([('Alice', 85), ('Bob', 70), ('Charlie', 60)])  
  
# Cada elemento es una tupla (clave, valor)
```

Iterar un diccionario

```
notas = {"Alice": 85, "Bob": 70, "Charlie": 60}

# Iterar sobre las claves
for clave in notas:
    print(clave)

# Iterar sobre los valores
for valor in notas.values():
    print(valor)

# Iterar sobre los pares clave-valor
for clave, valor in notas.items():
    print(f"{clave}: {valor}")
```

Ejemplo completo: diccionario de contactos

```
contactos = {  
    "Ana": "11-1234-5678",  
    "Luis": "11-8765-4321",  
    "Marta": "11-1111-2222"  
}  
  
# Buscar un contacto  
nombre = input("¿A quién buscás? ")  
telefono = contactos.get(nombre, "Contacto no encontrado")  
print(telefono)  
  
# Listar todos  
for nombre, tel in contactos.items():  
    print(f"{nombre}: {tel}")
```



Debug — Introducción

¿Qué es el debugging?

El **debugging** es el proceso de **identificar, analizar y corregir errores** en un programa.

- Asegura que el programa funcione correctamente y produzca los resultados esperados.
- Implica un proceso sistemático para localizar el origen del problema y aplicar una solución.



El término "bug" (insecto) tiene historia: en 1947 se encontró una polilla real atascada en un relé del Mark II, causando un fallo en la computadora.

Aspectos clave del debugging

1. **Identificar el problema:** observar comportamiento incorrecto, salidas inesperadas o mensajes de error.
2. **Aislar el problema:** acotar el área del código donde ocurre el error.
3. **Analizar la causa:** examinar la lógica, el flujo de datos y el estado del programa.
4. **Corregir el bug:** modificar el código, ajustar la lógica o aplicar un parche.
5. **Testear la corrección:** verificar que el problema se resolvió y que no se introdujeron nuevos errores.

Tipos de errores en Python

Tipo	¿Cuándo ocurre?	¿Da mensaje de error?
Error de Sintaxis	Al escribir código inválido	✓ Sí, antes de ejecutar
Error en tiempo de ejecución	Al ejecutar el programa	✓ Sí, durante la ejecución
Error lógico	El programa corre sin problemas	✗ No (resultado incorrecto)

Error de Sintaxis

Ocurre cuando el código **no sigue la gramática de Python**. Python no puede interpretar el código.

```
# Uso incorrecto de print (Python 2 style en Python 3)
print "Hola, mundo!"
```

```
File "<stdin>", line 1
  print "hola"
  ^^^^^^^^^^^
```

```
SyntaxError: Missing parentheses in call to 'print'.
Did you mean print(...)?
```

✓ **Cómo corregirlo:** `print("Hola, mundo!")`

Error en Tiempo de Ejecución (Excepción)

El código es **sintácticamente correcto**, pero falla al ejecutarse. También llamado **excepción**.

```
# Intentar dividir por cero  
resultado = 10 / 0
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ZeroDivisionError: division by zero
```

Otros ejemplos: acceder a un índice fuera de rango, convertir un string inválido a entero, abrir un archivo que no existe.

Error Lógico

El código **corre sin errores**, pero produce un **resultado incorrecto** por una falla en la lógica.

```
# Programa para calcular el promedio de dos números
def calcular_promedio(num1, num2):
    total = num1 + num2
    promedio = total / 3 # ← Error: debería dividir por 2
    return promedio

resultado = calcular_promedio(5, 10)
print(f"El promedio es: {resultado}")
# Imprime 5.0, pero debería ser 7.5
```

⚠ Son los más difíciles de detectar porque no hay mensaje de error.

Herramientas de debugging

1. `print()`

La herramienta más simple: imprimir valores intermedios para inspeccionar el estado del programa.

```
def calcular(x, y):  
    print(f"DEBUG: x={x}, y={y}") # Inspección  
    resultado = x * y + 10  
    print(f"DEBUG: resultado={resultado}")  
    return resultado
```

2. `logging`

Similar a `print()` pero con niveles (DEBUG, INFO, WARNING, ERROR) y más control.

3. IDE (PyCharm, VSCode)

Iterables — ¿Qué es un iterable?

Un **iterable** es un objeto capaz de devolver sus elementos de a uno por vez.

- Permite ser recorrido con un bucle `for`.
- Se puede **iterar** sobre sus elementos.

```
# Todos estos son iterables:
for letra in "Python":          # str
    print(letra)

for numero in [1, 2, 3]:        # list
    print(numero)

for valor in range(5):          # range
    print(valor)
```

Colecciones e iterables

Toda **colección** es un iterable. En Python las colecciones principales son:

- `list` — lista
- `tuple` — tupla
- `set` — conjunto
- `dict` — diccionario

Otros iterables que no son colecciones:

- `str` — cadenas de texto
- `range` — rango numérico
- `enumerate` — enumeración
- archivos (objetos de archivo)

Tabla comparativa de colecciones

Tipo	Ordenado	Mutable	Duplicados	Indexable	Ejemplo
list	✓	✓	✓	✓	[1, 2, 3]
tuple	✓	✗	✓	✓	(1, 2, 3)
set	✗	✓	✗	✗	{1, 2, 3}
dict	✓	✓	✓*	✓	{'a': 1}

*Los valores pueden repetirse, pero las **claves** no.

La función `range()`

`range()` genera una serie de números de forma eficiente.

```
# range(stop) – desde 0 hasta stop-1
for valor in range(5):
    print(valor)    # 0, 1, 2, 3, 4

# range(start, stop) – desde start hasta stop-1
for valor in range(1, 5):
    print(valor)    # 1, 2, 3, 4

# range(start, stop, step) – con paso personalizado
for valor in range(0, 10, 2):
    print(valor)    # 0, 2, 4, 6, 8
```

⚠ El límite superior (**stop**) es **excluido** — comportamiento "off-by-one".

`range()` — Convertir a lista

Se puede convertir la salida de `range()` a una lista con `list()`:

```
numeros = list(range(1, 6))
print(numeros)
# [1, 2, 3, 4, 5]

numeros_pares = list(range(2, 11, 2))
print(numeros_pares)
# [2, 4, 6, 8, 10]

# Generar los primeros 10 cuadrados perfectos
cuadrados = []
for valor in range(1, 11):
    cuadrados.append(valor ** 2)
print(cuadrados)
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

La función `enumerate()`

`enumerate()` recorre un iterable y **asigna un contador** a cada elemento.

```
nombres = ["Alice", "Bob", "Charly"]
```

```
for i, nombre in enumerate(nombres):  
    print(f"Índice {i}: {nombre}")
```

```
# Índice 0: Alice  
# Índice 1: Bob  
# Índice 2: Charly
```

```
# Se puede cambiar el inicio del contador  
for i, nombre in enumerate(nombres, start=1):  
    print(f"{i}. {nombre}")  
# 1. Alice 2. Bob 3. Charly
```

List Comprehension — Sintaxis

Una **list comprehension** crea listas en una sola línea combinando el `for` y la creación de elementos.

```
# Sintaxis básica  
lista = [expresion for elemento in iterable if condicion]
```

- La parte `if condicion` es **opcional**.

```
# Forma tradicional (4 líneas)  
cuadrados = []  
for x in range(1, 11):  
    cuadrados.append(x ** 2)  
  
# Con list comprehension (1 línea)  
cuadrados = [x**2 for x in range(1, 11)]  
print(cuadrados)
```

List Comprehension — Ejemplos

```
# Cuadrados del 1 al 10
cuadrados = [x**2 for x in range(1, 11)]
print(cuadrados)
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# Función matemática:  $x^2 + 2x + 3$ 
funcion = [x**2 + 2*x + 3 for x in range(1, 11)]
print(funcion)
# [6, 11, 18, 27, 38, 51, 66, 83, 102, 123]

# Solo números pares del 1 al 20
pares = [x for x in range(1, 21) if x % 2 == 0]
print(pares)
# [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Resumen de la clase

Diccionarios

- Estructura clave-valor, claves únicas
- Operaciones: acceder `[]`, agregar/actualizar `=`, eliminar `del` / `pop()`
- Métodos: `keys()`, `values()`, `items()`

Debug

- Tipos de error: **Sintaxis**, **Tiempo de ejecución**, **Lógico**
- Herramientas: `print()`, `logging`, IDE

Herramientas Iterables

- `range(start, stop, step)` — series numéricas
- `enumerate()` — índice + valor